Certification of Imitation Controllers Using Learned Lyapunov Functions

Master thesis by Josua Christoph Lindemann
Date: July 10, 2026, Date of submission: 16. August 2026

1. Review: Prof. Dr.-Ing. Rolf Findeisen
2. Review: Hendrik Alsmeier M.Sc.
Darmstadt

TECHNISCHE UNIVERSITÄT DARMSTADT

CCPS Control and Cyber-Physical Systems

Electrical Engineering and Information Technology Department

Certification of Imitation Controllers Using Learned Lyapunov Functions

Master thesis by Josua Christoph Lindemann

1. Review: Prof. Dr.-Ing. Rolf Findeisen
2. Review: Hendrik Alsmeier M.Sc.

Date: July 10, 2026
Date of submission: 16. August 2026

Darmstadt

Technische Universität Darmstadt
Institut für Automatisierungstechnik und Mechatronik
Fachgebiet Control and Cyber-Physical Systems
Prof. Dr.-Ing. Rolf Findeisen

Contents

1 Abstract

2 Introduction

3 Optimal Control and Lyapunov Stability Theory

3.1 Discrete-Time Dynamical Systems and Stability

To implement an optimal control scheme on digital hardware, the underlying continuous-time system dynamics

$$\dot{x}(t) = f(x(t), u(t)) \tag{3.1}$$

must be discretized, where $x(t) \in \mathcal{X}$ and $u(t) \in \mathcal{U}$ denote the continuous-time state and input vectors, respectively. In this work, the corresponding discrete-time system

$$x_{k+1} = f_d(x_k, u_k) \tag{3.2}$$

is obtained by approximating the continuous dynamics using a numerical integration scheme with a constant sampling time T_s, such as the forward Euler or a fourth-order Runge-Kutta (*RK4*) method **something**.

3.1.1 Lyapunov Stability

Let the origin $(x, u) = (\mathbf{0}, \mathbf{0})$ be an equilibrium point of the system such that $f_d(\mathbf{0}, \mathbf{0}) = \mathbf{0}$. Suppose the system is controlled by a state-feedback control policy $u_k = \kappa(x_k)$. The closed-loop system is said to be *Lyapunov stable* if there exists a positive definite Lyapunov function $V(x_k)$ such that:

$$\begin{aligned} V(\mathbf{0}) &= 0 & \\ V(x_k) &> 0 & \forall x_k \in \mathcal{X} \setminus \{\mathbf{0}\} \\ V(f_d(x_k, \kappa(x_k))) - V(x_k) &\leq 0 & \forall x_k \in \mathcal{X} \setminus \{\mathbf{0}\}. \end{aligned} \tag{3.3}$$

Furthermore, the equilibrium point is *locally asymptotically stable* if the Lyapunov function additionally satisfies a strictly negative definite decrease condition along the closed-loop trajectories:

$$V(f_d(x_k, \kappa(x_k))) - V(x_k) < 0 \qquad \forall x_k \in \mathcal{X} \setminus \{\mathbf{0}\}. \tag{3.4}$$

Asymptotic stability is typically the desired property for control systems, as it ensures that the system will converge to the equilibrium point over time.

3.2 Model Predictive Control and Stability

This section provides a brief overview of the Model Predictive Control (*MPC*) framework and its stability properties. First, the general MPC problem is formulated, followed by a discussion on the terminal set and terminal cost constraints. Afterwards, the closed-loop stability of the MPC scheme is analyzed.

3.2.1 Problem Formulation

For the nominal MPC setup considered in this work, the following finite-horizon optimal control problem is solved at each time step k given the current system state x_k:

$$\begin{aligned} \min_{\{x_i\}_{i=0}^{N},\{u_i\}_{i=0}^{N-1}} \quad & \sum_{i=0}^{N-1} \ell(x_i, u_i) + V_f(x_N) \\ \text{s.t.} \quad & x_{i+1} = f_d(x_i, u_i), && i = 0, \dots, N-1, \\ & x_i \in \mathcal{X}, && i = 0, \dots, N, \\ & u_i \in \mathcal{U}, && i = 0, \dots, N-1, \\ & x_0 = x_k. \end{aligned} \tag{3.5}$$

Here, N denotes the prediction horizon, $\ell(x_i, u_i)$ is the stage cost function, and $V_f(x_N)$ represents the terminal cost function. The system dynamics are represented by the discrete-time function $f_d(x_i, u_i)$, while $\mathcal{X}$ and $\mathcal{U}$ denote the state and input constraint sets, respectively.

A quadratic cost function is used for the stage cost, defined as

$$\ell(x_i, u_i) = x_i^\top Q x_i + u_i^\top R u_i, \tag{3.6}$$

where $Q \succeq 0$ and $R \succ 0$ are the state and input weighting matrices, respectively. Following the receding horizon principle, only the first control input u_0^* from the optimal sequence is applied to the system, and the optimization is repeated at the next sampling instance.

3.2.2 Terminal Ingredients for Stability

To ensure stability of a closed-loop system of a nominal MPC, a terminal set $\mathcal{X}_f \subseteq \mathcal{X}$ and a terminal cost function $V_f(x_N)$ are introduced. The terminal set $\mathcal{X}_f$ is a positively invariant set for the system under a local control law $u = \mu(x_k)$, and the terminal cost $V_f(x_N)$ is a Lyapunov function for the system within this set. However, to include these terminal ingredients, Eq. (3.5) must be augmented with the following terminal constraint:

$$x_N \in \mathcal{X}_f. \tag{3.7}$$

The property of a positive invariant set $\mathcal{X}_f$ is defined as a set of states where a state $x_k \in \mathcal{X}_f$ remains in $\mathcal{X}_f$ for all future time steps under the local control law $\mu(x_k)$,

$$f_d(x, \mu(x)) \in \mathcal{X}_f \quad \forall x \in \mathcal{X}_f. \tag{3.8}$$

This predictive mechanism is visualized in Section 3.2.2. While the initial state x_0 and the intermediate predicted states x_1, x_2 are only restricted to the safe state space $\mathcal{X}$, the terminal constraint Eq. (3.7) forces the final predicted state x_N to terminate strictly inside the invariant terminal set $\mathcal{X}_f$. Together with the terminal cost $V_f(x_N)$, which acts as a local Lyapunov function inside $\mathcal{X}_f$, these ingredients establish a mathematical bridge.

For nonlinear systems, the terminal set $\mathcal{X}_f$ is typically computed using linearized system dynamics around the equilibrium point $(x, u) = (\mathbf{0}, \mathbf{0})$, yielding the linear discrete-time state-space model:

$$x_{k+1} = Ax_k + Bu_k. \tag{3.9}$$

A widely used choice for these terminal ingredients is then derived from the Linear-Quadratic Regulator (*LQR*). Here, the local control law $\mu(x_k)$ is chosen as a linear state-feedback policy $\mu(x_k) = -Kx_k$, where the optimal LQR gain K is given by

$$K = \left(R + B^\top PB\right)^{-1} B^\top PA. \tag{3.10}$$

The matrix $P \succ 0$ is obtained as the unique positive definite solution to the discrete-time Algebraic Riccati Equation (*DARE*):

$$P = A^\top PA - A^\top PB \left(R + B^\top PB\right)^{-1} B^\top PA + Q. \tag{3.11}$$

This matrix parameterizes the quadratic terminal cost function $V_f(x_N) = x_N^\top P x_N$. Accordingly, the terminal set $\mathcal{X}_f$ is defined as an ellipsoidal sub-level set of this cost function:

$$\mathcal{X}_f = \left\{ x \in \mathbb{R}^n \;\middle|\; x^\top Px \leq \alpha \right\}. \tag{3.12}$$

To satisfy the positive invariance condition Eq. (3.8) for the original nonlinear system, the scaling parameter $\alpha > 0$ must be chosen sufficiently small. This ensures not only that all states within $\mathcal{X}_f$ satisfy the state constraints $\mathcal{X}$ and input constraints $\mathcal{U}$, but also that the local quadratic Lyapunov descent condition dominates the higher-order linearization errors of the nonlinear dynamics.

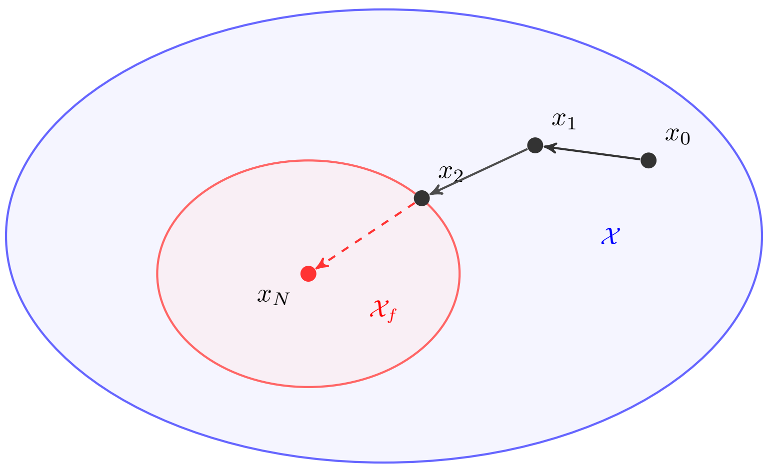

Figure 3.1: Comparison of the terminal set $\mathcal{X}_f$ (red) with the entire state space $\mathcal{X}$ (blue).

3.2.3 Closed-Loop Stability

TODO: ADD FORMULATION FOR RELAXED MPC STABILITY CONDITION

$$V_N(f_d(x_k, \mu(x_k))) \leq V_N(x_k) - \alpha_N \cdot \ell(x_k, \mu(x_k)), \quad \forall x_k \in \mathcal{X}_f \tag{3.13}$$

3.3 Stability without Terminal Constraints

only for no terminal constraints -> not asymptotically stable, but can be shown to be practically stable

$$\alpha_N = 1 - \frac{(\gamma - 1)^N}{\gamma^{N-1} - (\gamma - 1)^N} \tag{3.14}$$

if $\alpha_N > 0$ then

$$N < 2 + \frac{\ln(\gamma - 1)}{\ln(\gamma) - \ln(\gamma - 1)}, \tag{3.15}$$

then holds

$$V_N(x_k) \leq \gamma \ell^*(x_k) \tag{3.16}$$

4 Foundations of Neural Control Policies

4.1 Artificial Neural Networks

4.1.1 Multi-Layer Perceptrons

4.1.2 Optimization and Training

4.2 Counterexample-Guided Inductive Synthesis

5 Formal Verification of Neural Control Policies

5.1 Mixed-Integer Linear Programming

5.2 Branch and Bound

5.3 $\alpha\beta$-CROWN

6 Learning Lyapunov-Stable Imitation Controllers

6.1 Neural Policy Approximation and Imitation Learning

6.1.1 Data Generation and MPC Expert

6.1.2 Policy Network Architecture and Imitation Objective

6.2 Lyapunov Learning for Stability Certification

To achieve formal stability with learned controllers, a Lyapunov candidate is learned to certify stability. For this purpose, a neural network, as described in Section 4.1.1, is trained to approximate a Lyapunov candidate. The Counterexample-Guided Inductive Synthesis (*CEGIS*) framework is used to find stability violations and therefore make the Lyapunov candidate a valid certificate for stability.

6.2.1 Lyapunov Network Architecture

Ensuring that the learned Lyapunov candidate is positive definite is crucial for stability certification. To achieve this, a neural network architecture is designed to guarantee positive definiteness by construction, following the approach in **lyapunov-stable-nn**. The architecture combines the flexibility of neural networks while ensuring the mathematical properties required for Lyapunov functions. Resulting from this design, the Lyapunov candidate is defined as:

$$V_\phi(x_k) = |f_\phi(x_k) - f_\phi(x^*)| + \left\|(\epsilon I + R^\top R)(x_k - x^*)\right\|_1, \tag{6.1}$$

where $f_\phi : \mathbb{R}^n \to \mathbb{R}$ is a neural network, $x_k \in \mathbb{R}^n$ is the current state, $x^* \in \mathbb{R}^n$ is the equilibrium, $\epsilon > 0$ is a small regularization constant, and $R \in \mathbb{R}^{n \times n}$ is a learnable matrix. While the first term is only positive semi-definite and may vanish away from the equilibrium, the second term guarantees strict positive definiteness $\forall x_k \in \mathcal{X} \setminus \{x^*\}$, as the matrix $(\epsilon I + R^\top R)$ is strictly positive definite by construction for any $\epsilon > 0$.

6.2.2 Loss Formulation and CEGIS Training

To ensure that the Lyapunov candidate described in Eq. (6.1) can be certified for stability, the loss function must comprise multiple components. It utilizes the ReLU function $[\cdot]_+$ for specific penalties, alongside a weighted mean to aggregate the sample losses. Throughout the entire loss formulation, we consider a target sublevel value ρ, which is constrained by a lower bound $\rho_{\text{min}} > 0$ to ensure numerical stability.

Relative Lyapunov Decrease Loss is designed to ensure that the Lyapunov candidate decreases along the system trajectories. This is achieved by minimizing the relative difference between the Lyapunov candidate evaluated at the current state and the next state. For this purpose, the per-sample Lyapunov decrease term is defined as

$$e_V = V(x_{k+1}) - (1-\kappa) \cdot V(x_k) + \epsilon_m, \tag{6.2}$$

where $\kappa \in (0,1)$ is a hyperparameter that controls the desired rate of decrease, and $\epsilon_m \geq 0$ is a safety margin. To balance the optimization scale across different magnitudes of V, we normalize this violation to obtain the relative decrease loss:

$$\mathcal{L}_{\text{dec}} = \left[\frac{e_V}{|V(x_k)| + \epsilon_{\text{rel}}} \right]_+, \tag{6.3}$$

where $\epsilon_{\text{rel}} > 0$ prevents division by zero.

Relative State Invariance Loss penalizes trajectories that escape the training state space $\mathcal{B} = \{x \in \mathbb{R}^n \mid \underline{x} \leq x \leq \bar{x}\}$. Therefore, the relative one-step state-constraint violation loss for the next state x_{k+1} across all dimensions $j = 1, \dots, n$ is formulated per sample as

$$\mathcal{L}_{\text{inv}} = \sum_{j=1}^{n} \frac{[x_{k+1,j} - \bar{x}_j]_+ + [\underline{x}_j - x_{k+1,j}]_+}{\bar{x}_j - \underline{x}_j}. \tag{6.4}$$

Combined ρ-Gated Condition Loss is the sample-wise ρ-gated combination of the relative Lyapunov decrease loss from Eq. (6.3) and the relative state invariance loss from Eq. (6.4),

$$\mathcal{L}_{\text{cond}} = \mathcal{L}_{\text{dec}} + \omega_{\text{inv}} \cdot \mathcal{L}_{\text{inv}}, \tag{6.5}$$

where ω_{inv} is a hyperparameter balancing the relative importance of the two loss components. However, to apply the ρ-gate appropriately, $\mathcal{L}_{\text{cond}}$ is weighted for each sample using the soft sublevel weight

$$w_\rho = \text{sigmoid}\left(s \cdot \frac{\rho - V(x_k)}{\max(\rho, \epsilon_{\text{rel}})} \right), \tag{6.6}$$

where $s > 0$ denotes the gate sharpness. This smooth formulation ensures that the weights inside the ρ-sublevel are ≈ 1, whereas the weights for states far outside of ρ approach 0. The weighted mean of the combined condition loss $\mathcal{L}_{\text{cond}}$ and ρ-gating weight w_ρ results in the final loss formulation:

$$\mathcal{L}_{\rho\text{-gated}} = \frac{\sum_{i=1}^{N} w_\rho^{(i)} \cdot \mathcal{L}_{\text{cond}}^{(i)}}{\sum_{i=1}^{N} w_\rho^{(i)}} \tag{6.7}$$

ROA-Surrogate Loss tries to enlarge the region of attraction by creating samples within the boundary layer of the training space, defined as $\mathcal{B}_{\text{boundary}} = \mathcal{B} \setminus 0.6\mathcal{B}$. For a batch of M samples $x_{\text{ROA}}^{(i)} \in \mathcal{B}_{\text{boundary}}$ are generated and evaluated in the following formulation:

$$\mathcal{L}_{\text{ROA}} = \frac{1}{M} \sum_{i=1}^{M} \left[\frac{V(x_{\text{ROA}}^{(i)})}{\rho} - 1 \right]_{+} . \tag{6.8}$$

LIRPA-Condition Loss makes the Lyapunov function more verifier-friendly by including formal violations of the Lyapunov decrease condition as defined in Eq. (6.2), which are found using the LIRPA framework **lirpabook**. In order to compute these violations efficiently, the LIRPA framework is provided with a filtered subset of regions. This subset holds regions where either the center x_{center}, lower bounds x_{lower} or upper bounds x_{upper} satisfy $V(x) \leq \rho$. The model used for the LIRPA evaluation is the signed Lyapunov decrease loss $-e_V$ as given in Eq. (6.2). This is done such that LIRPA can provide a lower bound s_{lower} for the Lyapunov decrease condition, which inturn is used to define the LIRPA-condition loss. After formally evaluating the signed Lyapunov decrease lower bound s_{lower} for all N_{L} regions, the LIRPA-condition loss is defined as:

$$\mathcal{M}_{\text{L}} = \frac{[s_{\text{lower}}]_{+}}{V(x_{\text{center}}) + \epsilon_{\text{rel}}} . \tag{6.9}$$

The final LIRPA-condition loss is then computed as the weighted mean of all evaluated regions, with the weight formulated as the squared distance between the region center x_{center} and the equilibrium x^*:

$$w_L = \exp\left(-\alpha_L \cdot \sum_{j=0}^{n} \left(x_{\text{center},j} - x_j^* \right)^2 \right) , \tag{6.10}$$

where $\alpha_L > 0$ is a hyperparameter controlling the decay rate of the weight with respect to the distance from the equilibrium. Finally, the LIRPA-condition loss for N_{L} regions is computed as:

$$\mathcal{L}_{\text{LIRPA}} = \frac{\sum_{i=1}^{N_{\text{L}}} w_L^{(i)} \cdot \mathcal{M}_{\text{L}}^{(i)}}{\sum_{i=1}^{N_{\text{L}}} w_L^{(i)}} . \tag{6.11}$$

6.3 Formal Verification Methodology

7 Experimental Evaluation and Certification Results

7.1 System Modeling and Problem Formulation

For our experimental setup, two benchmark systems are selected: the double integrator and the cartpole. These systems provide a good balance between simplicity and complexity allowing us to certify the stability of the closed-loop system.

7.1.1 Double Integrator

The double integrator is a simple second-order linear system; therefore, it serves as the ideal baseline system in the learning and certification pipeline. Discretizing the continuous-time dynamics using the explicit Euler method with a sampling time of $\Delta t = 0.1$, leads to the discrete-time state-space equation:

$$x_{k+1} = f_{\mathrm{DI},d}(x_k, u_k) = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} x_k + \begin{bmatrix} 0 \\ \Delta t \end{bmatrix} u_k \tag{7.1}$$

where $x_k = \begin{bmatrix} p_k & v_k \end{bmatrix}^\top$ is the state vector containing the position and velocity of the system, respectively, and u_k denotes acceleration.

The *MPC setup* is formulated according to the problem in Eq. (3.5) subject to the double-integrator dynamics in Eq. (7.1). A prediction horizon of $N = 20$ steps is chosen, and the states and inputs are penalized using the weighting matrices $Q = \mathrm{diag}(15, 1)$ and $R = 0.1$.

7.1.2 Cartpole

The inverted pendulum on a cart, commonly referred to as the cartpole system, is used as a nonlinear benchmark system in this work. There are four states in the system: the cart position p, the cart velocity v, the pendulum angle θ, and the pendulum angular velocity ω. Here, the state vector is defined as $x = \begin{bmatrix} p & v & \theta & \omega \end{bmatrix}^\top$, and the input u is the force F applied to the cart. This leads to the following continuous-time dynamics of the cartpole system:

$$f_{\text{CP}}(x,u) = \begin{bmatrix} \dot{p} \\ \dot{v} \\ \dot{\theta} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} v \\ \frac{F+ml\omega^2 \sin(\theta)-mg\sin(\theta)\cos(\theta)}{M+m-m\cos^2(\theta)} \\ \omega \\ \frac{-F\cos(\theta)-ml\omega^2\sin(\theta)\cos(\theta)+(M+m)g\sin(\theta)}{l(M+m-m\cos^2(\theta))} \end{bmatrix} \tag{7.2}$$

where $M = 1\,\text{kg}$ is the mass of the cart, $m = 0.1\,\text{kg}$ is the mass of the pendulum, $l = 0.8\,\text{m}$ is the length of the pendulum, and $g = 9.81\,\text{m/s}^2$ is the acceleration due to gravity. Discretizing the continuous-time dynamics using the explicit Euler method with a sampling time of $\Delta t = 0.05\,\text{s}$, leads to the discrete-time state-space equation:

$$x_{k+1} = f_{\text{CP},d}(x_k, u_k) = x_k + \Delta t f_{\text{CP}}(x_k, u_k) \tag{7.3}$$

For the nonlinear benchmark, a prediction horizon of $N = 40$ stages is chosen. The system states and control inputs are penalized within the optimization framework of Eq. (3.5) using the weighting matrices $Q = \text{diag}(10, 1, 10, 0.01)$ and $R = 0.5$, respectively, subject to the dynamics defined in Eq. (7.3).

7.2 Linear System Results: Double Integrator

7.3 Nonlinear System Results: Cartpole

8 Discussion

8.1 Qualitative Discussion of Results

8.2 Challenges with Nonlinear Dynamics

8.3 Scalability and Limitations

9 Conclusion

A Appendix